

DATA PREPROCESSING

Feature Scaling

Feature scaling is a crucial preprocessing step in many machine learning workflows. It ensures that numerical features contribute equally to model performance and prevents features with larger magnitudes from dominating those with smaller ones. Below are the commonly used feature scaling techniques:

1. Standardization

- **Formula:** $x' = \frac{x - \mu}{\sigma}$, where μ is the mean and σ is the standard deviation of the feature.
- This method transforms the data to have a mean of 0 and a standard deviation of 1.
- It is particularly effective for algorithms that assume normally distributed features, such as linear regression, logistic regression, and principal component analysis (PCA).

2. Normalization

- **Min-Max Scaling:**
 - **Formula:** $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$
 - This technique scales the data to a fixed range, typically $[0, 1]$.
 - It is suitable for cases where the presence of both positive and negative values is known and bounded.
- **Max-Abs Scaling:**
 - **Formula:** $x' = \frac{x}{\max(|x|)}$
 - This method scales each feature by its maximum absolute value, resulting in values between -1 and 1.
 - It is especially effective for sparse data representations.
- **Robust Scaling:**
 - **Formula:** $x' = \frac{x - \text{median}(x)}{\text{IQR}}$, where IQR is the interquartile range.
 - This method is less sensitive to outliers as it uses median and IQR instead of mean and standard deviation.
 - It is preferred when the dataset contains significant outliers.

Encoding Categorical Data

Encoding categorical variables is essential to convert non-numeric data into a numerical format that can be interpreted by machine learning algorithms. The two primary techniques are described below:

1. Ordinal Encoding

This technique is used when categorical features have an inherent order or ranking. Each category is assigned an integer value reflecting its relative position.

Original Category	Encoded Value
Poor	0
Good	1
Excellent	2

2. Nominal Encoding (One-Hot Encoding)

This method is applied when categories do not have any inherent order. Each category is transformed into a binary vector where only one bit is set to 1, indicating the presence of that category.

Pet Type	Dog	Cat	Rabbit
Dog	1	0	0
Cat	0	1	0
Rabbit	0	0	1

Mathematical Transformations

Mathematical transformations are applied to features to stabilize variance, normalize distributions, and improve the performance of machine learning algorithms.

1. Function Transformations

These transformations apply mathematical functions directly to the feature values to address skewness and non-linearity.

- **Reciprocal Transformation:** $y = \frac{1}{x}$
Effective for reducing the impact of large values.
- **Square Transformation:** $y = x^2$
Useful for handling **left-skewed data** by stretching values on the right.
- **Logarithmic Transformation:** $y = \log(x + 1)$ or $y = \log(x)$
Appropriate for compressing **right-skewed data** and reducing the influence of large values.
- **Square Root Transformation:** $y = \sqrt{x}$
Less aggressive than log transformation, often used for count data.

2. Power Transformations

Power transforms are parametric methods that aim to make data more Gaussian-like.

- **Box-Cox Transformation:**

– Defined as:

$$x' = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \log(x), & \lambda = 0 \end{cases}$$

– **Note:** Applicable only to strictly positive data.

- **Yeo-Johnson Transformation:**

– Defined as:

$$y' = \begin{cases} \frac{(y+1)^\lambda - 1}{\lambda}, & \lambda \neq 0, y \geq 0 \\ \log(y + 1), & \lambda = 0, y \geq 0 \\ -\frac{(-y+1)^{2-\lambda} - 1}{2-\lambda}, & \lambda \neq 2, y < 0 \\ -\log(-y + 1), & \lambda = 2, y < 0 \end{cases}$$

– **Note:** Suitable for both positive and negative values.

3. Quantile Transformation

Quantile transformation maps the original data to a specified distribution using its empirical cumulative distribution function (CDF).

- **Overview:** Converts features into a uniform or normal distribution by ranking the values.
- **Steps Involved:**
 1. **Sort the Data:** Arrange values in ascending order.
 2. **Compute Rank:** For each data point x_i , calculate the rank as:

$$r_i = \frac{i}{n}$$

where i is the index in the sorted list and n is the total number of samples.

3. **Map to Target Distribution:**

- **Uniform Mapping:**

$$y_i = r_i = \frac{i}{n}$$

- **Normal Mapping:**

$$y_i = \Phi^{-1}(r_i)$$

where Φ^{-1} is the inverse cumulative distribution function (CDF) of the standard normal distribution.

4. **Handle Ties:** For duplicate values, assign the average of their ranks to ensure consistency.

Encoding Numerical Features

Encoding numerical features involves transforming continuous or discrete numeric values into formats that are more suitable for machine learning algorithms.

1. Discretization / Binning

Discretization transforms continuous numerical features into categorical values by grouping them into discrete intervals or bins.

- **Unsupervised Binning:** Bin boundaries are determined without using information from the target variable.
 - **Uniform Binning (Equal Width):** Divides the range of the variable into k equal-width intervals.
 - **Quantile Binning (Equal Frequency):** Ensures each bin has approximately the same number of observations.
 - **K-means Binning:** Uses the K-means clustering algorithm to determine bin centers based on feature distribution.
- **Supervised Binning:** Bin boundaries are chosen based on the relationship between the feature and the target variable (e.g., maximizing information gain or minimizing impurity).
- **Custom Binning:** Bin intervals are manually defined using domain expertise or data-specific knowledge.

2. Binarization

Binarization converts continuous numerical values into binary values (0 or 1) based on a predefined threshold.

- **Binary Encoding Rule:**

$$y_i = \begin{cases} 0, & \text{if } x_i \leq \text{threshold} \\ 1, & \text{if } x_i > \text{threshold} \end{cases}$$

- Often used to simplify features or prepare them for algorithms that expect binary input.

Outlier

Outlier Detection Techniques

1. Mean and Standard Deviation Method (For Normally Distributed Data)

- Lower Bound: $\mu - 3\sigma$
- Upper Bound: $\mu + 3\sigma$
- Any value outside this range is considered an outlier.

2. Interquartile Range (IQR) Method (For Skewed Data)

- $Q1 = 25\text{th percentile}$, $Q3 = 75\text{th percentile}$
- $IQR = Q3 - Q1$
- Lower Bound: $Q1 - 1.5 \times IQR$
- Upper Bound: $Q3 + 1.5 \times IQR$

- Values outside this range are considered outliers.

3. Percentile Method

- Outliers are defined based on extreme percentiles.
- Common thresholds: below the 1st percentile or above the 99th percentile.
- Useful for non-normal or highly skewed distributions.

Outlier Handling Techniques

1. Trimming (Removing Outliers)

- Outliers are removed from the dataset.
- Useful when outliers are likely due to data entry errors or noise.

2. Capping (Winsorizing)

- Outliers are replaced with the nearest boundary value (e.g., 1st or 99th percentile).
- Helps retain all data points while limiting the influence of extreme values.

Handling Missing Values

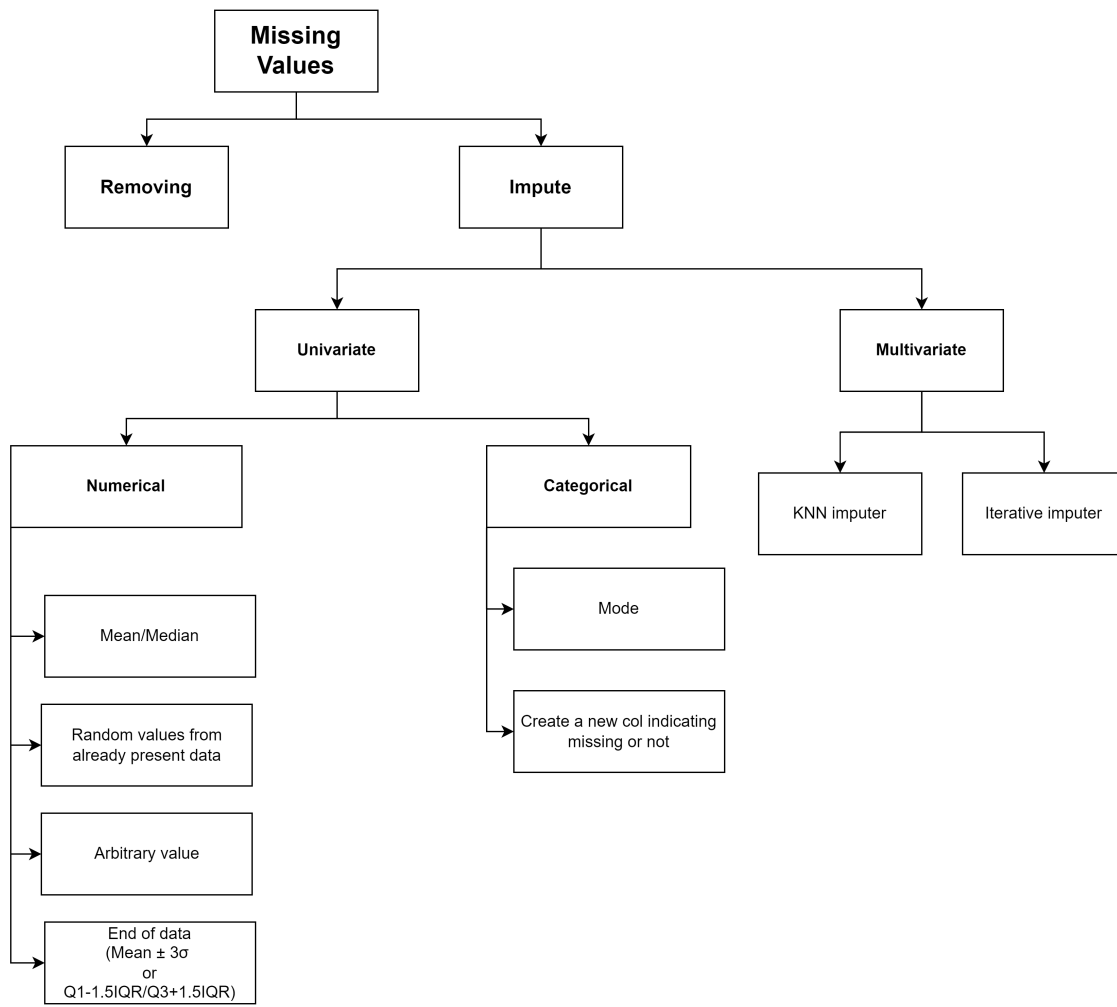


Figure 1: Handling Missing Values

Handling missing values is a critical preprocessing step. Strategies depend on the type, amount, and randomness of the missing data.

Removing

Rows with missing values can be removed under specific conditions to avoid introducing bias or information loss.

- **Missing Completely at Random (MCAR):** Ensure that the missing values are random and not dependent on any feature.
- **Low Missing Percentage:** Preferably less than 5% of the dataset.
- **Distribution Preservation:** Verify that the removal does not significantly affect the data distribution:
 - The probability density function (PDF) of numerical columns should remain similar.
 - The distribution of categorical values should also remain consistent.

KNN Imputer

Row	feature1	feature2	feature3
0	1.0	2.0	1.5
1	2.0	NaN	2.5
2	NaN	6.0	3.5
3	4.0	8.0	NaN
4	5.0	10.0	5.5

Step 1: Calculate Squared Euclidean Distance

We compute the squared Euclidean distance for each pair of rows using only the available (non-missing) values.

$$\text{Distance}(A, B) = \sum (\text{feature}_i^{(A)} - \text{feature}_i^{(B)})^2$$

	Row 0	Row 1	Row 2	Row 3	Row 4
Row 0	0	2	20	45	96
Row 1	2	0	1	34.25	18
Row 2	20	1	0	4	20
Row 3	45	34.25	4	0	5
Row 4	96	18	20	5	0

Step 2: Adjust Distance Using Missing Values

We modify the squared Euclidean distance to account for missing values using the formula:

$$\text{Adjusted Distance} = \sqrt{\text{Squared Distance} \times \frac{\text{Total Columns}}{\text{Columns Used}}}$$

where:

- **Total Columns** = 3 (feature1, feature2, feature3)
- **Columns Used** = Number of non-missing values used in distance computation.

	Row 0	Row 1	Row 2	Row 3	Row 4
Row 0	$\sqrt{0 \times \frac{3}{3}}$	$\sqrt{2 \times \frac{3}{2}}$	$\sqrt{20 \times \frac{3}{2}}$	$\sqrt{45 \times \frac{3}{2}}$	$\sqrt{96 \times \frac{3}{3}}$
Row 1	$\sqrt{2 \times \frac{3}{2}}$	$\sqrt{0 \times \frac{3}{3}}$	$\sqrt{1 \times \frac{3}{1}}$	$\sqrt{34.25 \times \frac{3}{1}}$	$\sqrt{18 \times \frac{3}{2}}$
Row 2	$\sqrt{20 \times \frac{3}{2}}$	$\sqrt{1 \times \frac{3}{1}}$	$\sqrt{0 \times \frac{3}{3}}$	$\sqrt{4 \times \frac{3}{2}}$	$\sqrt{20 \times \frac{3}{2}}$
Row 3	$\sqrt{45 \times \frac{3}{2}}$	$\sqrt{34.25 \times \frac{3}{1}}$	$\sqrt{4 \times \frac{3}{2}}$	$\sqrt{0 \times \frac{3}{3}}$	$\sqrt{5 \times \frac{3}{2}}$
Row 4	$\sqrt{96 \times \frac{3}{3}}$	$\sqrt{18 \times \frac{3}{2}}$	$\sqrt{20 \times \frac{3}{2}}$	$\sqrt{5 \times \frac{3}{2}}$	$\sqrt{0 \times \frac{3}{3}}$

	Row 0	Row 1	Row 2	Row 3	Row 4
Row 0	0.000	1.732	5.477	9.486	16.970
Row 1	1.732	0.000	1.732	10.143	5.196
Row 2	5.477	1.732	0.000	3.464	5.477
Row 3	9.486	10.143	3.464	0.000	2.738
Row 4	16.970	5.196	5.477	2.738	0.000

Step 3: Apply Uniform or Distance Method for Imputation

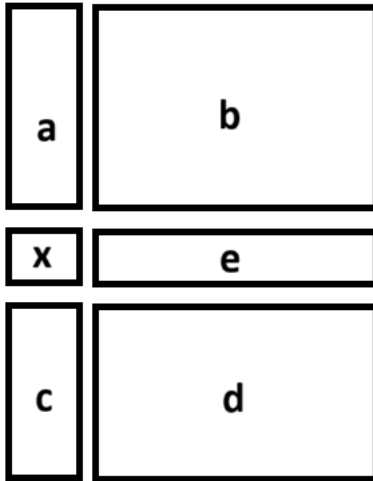
- **Uniform Method:** Uses the average of the feature values of the K nearest neighbors.
- **Distance Method:** Uses the weighted average where weights are inverse distances:

$$\text{Imputed Value} = \frac{\sum \left(\frac{1}{\text{dist}} \times \text{feature value} \right)}{\sum \left(\frac{1}{\text{dist}} \right)}$$

Row	feature1	feature2	feature3
0	1.0	2.0	1.5
1	2.0	$\frac{2.0}{1.732} + \frac{6.0}{5.196} + \frac{8.0}{10.143} + \frac{10.0}{16.970}$	2.5
2	$\frac{1.0}{5.477} + \frac{2.0}{1.732} + \frac{4.0}{3.464} + \frac{5.0}{5.477}$	6.0	3.5
3	4.0	8.0	$\frac{1.5}{9.486} + \frac{2.5}{10.143} + \frac{3.5}{3.464} + \frac{5.5}{2.738}$
4	5.0	10.0	5.5

Iterative Imputation

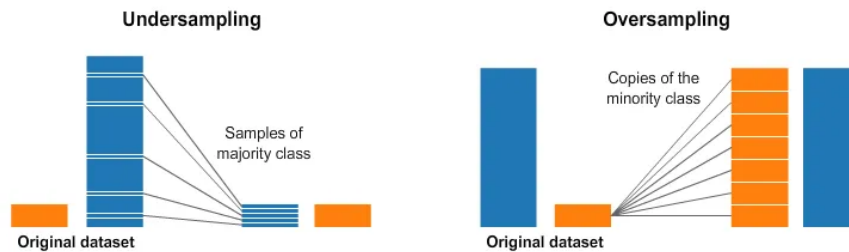
Iterative imputation is an advanced technique used to handle missing values in a dataset. Unlike simple imputation methods, such as filling missing values with the mean, iterative imputation attempts to predict missing values based on other features in the dataset using a model.



- **Initial Imputation:** Start by filling missing values (NaN) with the mean or median of the respective feature to provide a starting point for the imputation process.
- **Choose a NaN value (X)** to impute from the dataset.
- **Train a model** (e.g., linear regression, decision tree, etc.) where:
 - The features (inputs) for training the model are derived from other columns of the dataset. For example, the features could be a combination of columns *b* and *d*, denoted as (*b* + concat *d*).
 - The target variable (the value we want to predict) is a combination of the values from columns *a* and *c*, denoted as concat *a* + *c*. This target represents the missing value in column *X*.
- **Predict the missing value:** Once the model is trained, use the available data from other columns (e.g., column *e*) to predict the missing value *X*.
- **Repeat the process:** Repeat this process for all missing values in the dataset.
- **Update the missing values:** After imputing all the missing values in the dataset, update the missing values with the predicted values from the model.
- **Iterative updates:** Instead of using the mean for imputation in the next iteration, use the newly predicted values as the starting point for the next iteration of imputation.
- **Convergence criteria:** Repeat this process for several iterations or until the imputed values stabilize, meaning the changes between iterations become minimal.

Handling Imbalanced Data

1. Undersampling and Oversampling



2. SMOTE (Synthetic Minority Oversampling Technique)

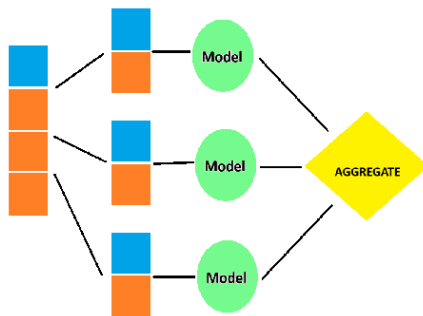
- **Train a k-NN model** on minority class observations:
 - Identify the **k nearest neighbors** for each minority class sample (commonly $k = 5$). These neighbors are used to generate synthetic examples that lie within the same feature space as the original data points.
- **Create synthetic data:**
 1. **Select 1 example** randomly from the minority class. This serves as the base for creating synthetic examples.
 2. **Select one neighbor** randomly from its k -nearest neighbors. This step ensures that the generated samples are close to the original data points in the feature space.
 3. Extract a **random number** α between 0 and 1 for interpolation. This value controls how much the synthetic sample will deviate from the original sample.
 4. Generate the synthetic sample using the formula:

$$\text{Synthetic sample} = \text{Original sample} + \alpha \times (\text{Neighbor} - \text{Original sample})$$

This step interpolates between the original sample and its neighbor, creating a new sample that lies in between.

- Repeat the process to create multiple synthetic samples for each minority class observation.
- **Combine the original dataset with synthetic samples** to form a balanced dataset. This helps ensure that the model is not biased towards the majority class.

3. Ensemble Methods



4. Giving Weights to Different Classes

- **Class weights** are applied to penalize misclassifications in the minority class. The more the class imbalance, the higher the weight is given to the minority class to make it more important during the model training process.
- This can be especially helpful in algorithms like *Logistic Regression* or *Support Vector Machines*, where the decision boundary is influenced by the weights given to each class.
- While applying class weights can help in improving model performance, it may require fine-tuning to balance the performance of both classes without overfitting to the minority class.

CLASSIFICATION METRICS

Metric	Formula / Definition	Interpretation
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Proportion of total predictions that were correct. Can be misleading on imbalanced datasets. Greater values indicate better performance.
Precision	$\frac{TP}{TP + FP} = \frac{\text{True Positive}}{\text{Predicted Positive}}$	Proportion of positive predictions that were truly positive. Greater values indicate better performance.
Recall (Sensitivity)	$\frac{TP}{TP + FN} = \frac{\text{True Positive}}{\text{Actual Positive}}$	Proportion of actual positive instances correctly identified. Greater values indicate better performance.
Specificity (True Negative Rate)	$\frac{TN}{TN + FP} = \frac{\text{True Negative}}{\text{Actual Negative}}$	Proportion of actual negative instances correctly identified. Greater values indicate better performance.
Balanced Accuracy	$\frac{\text{Recall} + \text{Specificity}}{2} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right)$	Average of recall for each class. Useful for imbalanced datasets as it balances TPR and TNR. Greater values indicate better performance.
MCC	$\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$	Measures the correlation between the observed and predicted binary classifications. Ranges from -1 (perfect inverse prediction) to +1 (perfect prediction). 0 indicates a random prediction. Useful for imbalanced data. Higher values indicate better performance.
Cohen's Kappa (κ)	$\kappa = \frac{P_o - P_e}{1 - P_e}$	Measures agreement between predicted and actual labels, correcting for chance agreement. P_o is observed agreement (Accuracy), P_e is expected agreement by chance. Ranges from -1 to +1. 0 means chance agreement, +1 is perfect agreement. Higher values indicate better performance.
F1-score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall. Balances both equally ($\beta = 1$). Greater values indicate better performance.
F β -score	$(1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$	Weighted harmonic mean of precision and recall. β weights recall relative to precision: $\beta > 1$ favors recall, $\beta < 1$ favors precision. Greater values indicate better performance.
Log Loss	$-\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$	Penalizes confident incorrect probabilistic predictions. Lower values indicate better performance.
Area Under ROC (AUC)	Area under the ROC curve (TPR vs FPR).	Represents the model's ability to distinguish between positive and negative classes. Greater values (closer to 1) indicate better performance.

ROC Curves

When evaluating machine learning classification models, especially binary classifiers, simply looking at accuracy can be misleading. The **Receiver Operating Characteristic (ROC) curve** is a powerful graphical tool that provides a more comprehensive view of a model's performance across all possible classification thresholds.

An ROC curve plots two key metrics against each other:

1. **True Positive Rate (TPR)** (Sensitivity or Recall): The proportion of actual positive instances correctly identified.

$$TPR = \frac{TP}{TP + FN} = \frac{TP}{P}$$

2. **False Positive Rate (FPR)**: The proportion of actual negative instances incorrectly identified as positive.

$$FPR = \frac{FP}{FP + TN} = \frac{FP}{N}$$

Where:

- **TP**: True Positive (Actual is Positive, Predicted is Positive)
- **FP**: False Positive (Actual is Negative, Predicted is Positive)
- **FN**: False Negative (Actual is Positive, Predicted is Negative)
- **TN**: True Negative (Actual is Negative, Predicted is Negative)
- **N**: Total Negative
- **P**: Total Positive

How the ROC Curve is Generated and Calculated: An Example

Most classification models output a probability or score for an instance belonging to the positive class. A decision threshold is used to convert this score into a binary prediction. The ROC curve is generated by varying this threshold from very high to very low. For each potential threshold, the TPR and FPR are calculated, and these (FPR, TPR) pairs are plotted.

Let's illustrate with a simple example. Suppose we have 10 instances, 5 actual positives (Spam=1) and 5 actual negatives (Not Spam=0). We have the model's predicted probability for each. To generate the ROC curve points, we sort the instances by their predicted probability (highest first) and then calculate the TP, FP, TPR, and FPR values for thresholds that incrementally include one or more additional instances as 'predicted positive'. Note that the total number of actual positives, P=5 and negatives, N=5.

The table below shows the sorted instances and the step-by-step calculation of the ROC points as the effective threshold is lowered, focusing on TP and FP which directly determine TPR and FPR. Total Actual Positives (P) = 5, Total Actual Negatives (N) = 5.

Sorted Instance ID	Actual Label	Predicted Prob	TP	FP	TPR ($\frac{TP}{P}$)	FPR ($\frac{FP}{N}$)	ROC Point
		1.0	0	0	0	0	(0.0, 0.0)
1	1	0.9	1	0	0.2	0	(0.0, 0.2)
2	0	0.8	1	1	0.2	0.2	(0.2, 0.2)
3	1	0.75	2	1	0.4	0.2	(0.2, 0.4)
4	1	0.7	3	1	0.6	0.2	(0.2, 0.6)
5	0	0.6	3	2	0.6	0.4	(0.4, 0.6)
6	1	0.55	4	2	0.8	0.4	(0.4, 0.8)
7	0	0.4	4	3	0.8	0.6	(0.6, 0.8)
8	1	0.3	5	3	1.0	0.6	(0.6, 1.0)
9	0	0.2	5	4	1.0	0.8	(0.8, 1.0)
10	0	0.1	5	5	1.0	1.0	(1.0, 1.0)

The ROC curve is plotted using the (FPR, TPR) pairs from the last column of the table above.

Interpreting the ROC Curve

- The x-axis (FPR) represents the rate of false alarms. It also represent till that point how much percentage of positive classes are identified of total positive class.
- The y-axis (TPR) represents the rate of correct positive predictions. It also repreent till that point how much negative class point are identified of total negative class.
- The ideal point is (0,1) - 0% false positives and 100% true positives.
- The diagonal line from (0,0) to (1,1) represents a random classifier. A good model's curve bows towards the top-left corner, staying as far from the diagonal as possible.

Area Under the Curve (AUC)

The **Area Under the ROC Curve (AUC)** is a single number that summarizes the overall performance depicted by the curve.

- AUC ranges from 0 to 1.
- An AUC of 1 indicates a perfect classifier.
- An AUC of 0.5 indicates a random classifier.
- An AUC less than 0.5 indicates performance worse than random.
- Higher AUC values generally indicate better model performance across various thresholds.

For our example points, the AUC can be calculated by summing the areas of the trapezoids between consecutive points (FPR_i, TPR_i) and (FPR_{i+1}, TPR_{i+1}) :

$$AUC = \sum_i (FPR_{i+1} - FPR_i) \times \frac{TPR_i + TPR_{i+1}}{2}$$

Calculating the AUC for our example points gives an AUC of **0.70**. This indicates the model performs better than random.

Cohen's Kappa

In evaluating classification models, while accuracy measures the overall proportion of correct predictions, it doesn't account for correct predictions that could happen purely by chance. This is where Cohen's Kappa (κ) provides a more robust and interpretable evaluation.

What is Cohen's Kappa?

Cohen's Kappa (κ) is a statistical measure used to evaluate the level of agreement between two raters(models) or between a model's predictions and the ground truth, correcting for the agreement that occurs randomly.

Why Cohen's Kappa? (Beyond Accuracy)

Accuracy is the ratio of correct predictions to total predictions. However, it can be misleading in imbalanced datasets. For example, if 95% of instances belong to class A, a model predicting only class A will still achieve 95% accuracy without meaningful classification.

Cohen's Kappa adjusts for chance agreement, offering a more reliable measure when class imbalance or random agreement is a concern.

Formula

Cohen's Kappa (κ) is defined as:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

Where:

- P_o is the **observed agreement (accuracy)**:

$$P_o = \frac{\sum_k n_{kk}}{N}$$

- P_e is the **expected agreement by chance**:

$$P_e = \frac{\sum_k n_{k.} \cdot n_{.k}}{N^2}$$

Here, n_{kk} is the number of correct predictions for class k , $n_{k.}$ is the total actual instances of class k , $n_{.k}$ is the total predicted instances of class k , and N is the total number of instances.

Interpretation Guidelines for κ

Cohen's κ ranges from -1 to 1:

- **+1**: Perfect agreement
- **0**: Agreement by chance
- **-1**: Complete disagreement

Interpretation (Landis Koch, 1977):

κ Value	Agreement Level
< 0	Less than chance
0.01–0.20	Slight
0.21–0.40	Fair
0.41–0.60	Moderate
0.61–0.80	Substantial
0.81–1.00	Almost perfect

Multiclass Classification Example

Let's consider a 3-class problem (A, B, C) with the following confusion matrix for 120 instances:

	Predicted A	Predicted B	Predicted C	Total
Actual A	40	5	5	50
Actual B	3	32	5	40
Actual C	2	3	25	30
Total	45	40	35	120

Total instances $N = 120$.

Step 1: Calculate Observed Agreement (P_o)

P_o is the sum of the diagonal elements divided by the total instances:

$$P_o = \frac{40 + 32 + 25}{120} \approx 0.8083$$

Step 2: Calculate Expected Agreement by Chance (P_e)

P_e is the sum, over each class, of the product of the actual total for that class and the predicted total for that class, divided by N^2 :

$$P_e = \frac{(50 \times 45) + (40 \times 40) + (30 \times 35)}{120 \times 120} \approx 0.3403$$

Step 3: Calculate Kappa (κ)

$$\kappa = \frac{P_o - P_e}{1 - P_e} \approx \frac{0.8083 - 0.3403}{1 - 0.3403} = \frac{0.4680}{0.6597} \approx 0.7095$$

According to the interpretation scale, a κ of ≈ 0.71 indicates **Substantial agreement**.

REGRESSION METRICS

Metric	Formula / Definition	Interpretation
Mean Absolute Error (MAE)	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Average absolute difference between predicted and actual values. Measures average error magnitude. Lower values indicate better performance.
Mean Squared Error (MSE)	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Average squared difference between predicted and actual values. Penalizes larger errors more heavily than MAE. Lower values indicate better performance.
Root Mean Squared Error (RMSE)	$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	Square root of MSE. In the same units as the response variable. Measures average error magnitude. Lower values indicate better performance.
R-squared (R^2)	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	Proportion of the variance in the dependent variable that is predictable from the independent variable(s). Ranges from 0 to 1. Higher values (closer to 1) indicate better fit.
Adjusted R-squared	$1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$	Modified R-squared that adjusts for the number of predictors (p) and observations (n). Penalizes the inclusion of irrelevant predictors. Useful for model comparison. Higher values indicate better performance.

DIMENSION REDUCTION

Principal Component Analysis (PCA)

- Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original features into a smaller set of new features called principal components. These components are orthogonal (uncorrelated) and capture the maximum variance present in the data. The goal of PCA is to reduce the number of features while retaining the most significant information, making the dataset easier to analyze, visualize, and model.
- The key idea behind PCA is to project the data onto directions that maximize the variance, thus helping uncover hidden patterns and reducing noise in the data.

Steps for Principal Component Analysis (PCA)

1. Mean Centering the Data:

- The first step is to center the data by subtracting the mean of each feature (column). This step ensures that the data has zero mean and allows PCA to focus on the variance rather than the absolute values of the features.
- Mathematically, for a feature f_i , subtract its mean, i.e., $f'_i = f_i - \mu_{f_i}$, where μ_{f_i} is the mean of feature f_i .

2. Compute the Covariance Matrix:

- The covariance matrix C captures the relationships between the features. If X is the matrix of mean-centered data, the covariance matrix is computed as:

$$C = \frac{1}{n-1} X^T X$$

where n is the number of samples and X^T is the transpose of the mean-centered data matrix.

- Each element in the covariance matrix represents how two features vary together. Diagonal elements represent the variance of each feature, and off-diagonal elements show the covariance between pairs of features.

	f_1	f_2	$\dots$	f_m
f_1	$\text{cov}(f_1, f_1)$	$\text{cov}(f_1, f_2)$	$\dots$	$\text{cov}(f_1, f_m)$
f_2	$\text{cov}(f_2, f_1)$	$\text{cov}(f_2, f_2)$	$\dots$	$\text{cov}(f_2, f_m)$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
f_m	$\text{cov}(f_m, f_1)$	$\text{cov}(f_m, f_2)$	$\dots$	$\text{cov}(f_m, f_m)$

3. Calculate Eigenvalues and Eigenvectors:

- Eigenvalues represent the magnitude of variance explained by each principal component, while eigenvectors represent the directions (or axes) along which the data varies the most.
- Solve for the eigenvalues (λ) and eigenvectors (v) of the covariance matrix C using the equation:

$$Cv = \lambda v$$

where v is the eigenvector and λ is the eigenvalue. The eigenvectors form the basis of the new feature space.

4. Select Principal Components:

- Sort the eigenvalues in descending order to identify the most important principal components that explain the most variance.
- Choose the top p eigenvectors corresponding to the largest eigenvalues. These eigenvectors represent the directions that capture the most significant variance in the data.
- Construct a matrix V containing the selected p eigenvectors. The dimensions of V will be $m \times p$, where m is the number of features, and p is the number of principal components chosen.

5. Project Data onto New Subspace:

- To reduce the dimensionality of the data, multiply the mean-centered data matrix X by the matrix V containing the selected eigenvectors:

$$X_{\text{new}} = X \cdot V$$

- This transformation projects the data onto the new subspace defined by the top p principal components. The resulting data matrix X_{new} will have n rows (same number of samples) and p columns (number of selected principal components).

6. Reconstruction (Optional):

- In some cases, you might want to reconstruct the original data using the reduced-dimensional data. This is done by multiplying the reduced data matrix by the transpose of the eigenvector matrix:

$$X_{\text{reconstructed}} = X_{\text{new}} \cdot V^T$$

- The reconstructed data will be an approximation of the original data, with the error proportional to the variance lost by discarding less significant components.

Linear Discriminant Analysis (LDA)

Given a dataset with c classes, let:

- $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]$ represent the dataset with n samples.
- Each sample $\mathbf{x}_i \in R^d$ is a d -dimensional feature vector.
- Let $\mathbf{X}_j$ be the subset of samples belonging to class j , which has n_j samples.

The main goal of LDA is to find a projection that maximizes the separability between different classes while minimizing the variance within each class.

Steps in Linear Discriminant Analysis (LDA)

1. Class Mean Vector:

- For each class j , calculate the mean of the samples:

$$m_j = \frac{1}{n_j} \sum_{x \in X_j} x$$

where n_j is the number of samples in class j .

2. Within-Class Scatter Matrix:

- This matrix captures the spread of each class around its own mean:

$$S_W = \sum_{j=1}^c \sum_{x \in X_j} (x - m_j)(x - m_j)^T$$

where x represents a sample and m_j is the class mean.

3. Between-Class Scatter Matrix:

- This matrix quantifies how far apart the class means are from the overall mean m of the dataset:

$$S_B = \sum_{j=1}^c n_j (m_j - m)(m_j - m)^T$$

where n_j is the number of samples in class j and m is the overall mean of the dataset.

4. Optimization Objective:

- The goal is to find a projection vector w that maximizes the ratio of between-class variance to within-class variance:

$$J(w) = \frac{w^T S_B w}{w^T S_W w}$$

This ratio is maximized when the classes are as far apart as possible while keeping the variance within each class as small as possible.

5. Eigenvalue Problem:

- Solve the generalized eigenvalue problem to find the projection vector:

$$S_B w = \lambda S_W w$$

where S_W and S_B are the within-class and between-class scatter matrices, respectively, and λ is the eigenvalue.

- Compute the eigenvalues and eigenvectors of the matrix $\mathbf{S}_W^{-1} \mathbf{S}_B$.
- Sort the eigenvectors in descending order based on their corresponding eigenvalues.
- Select the top k eigenvectors (where $k \leq c - 1$) to form the transformation matrix $\mathbf{W}$, where each eigenvector corresponds to a principal axis in the new subspace.

Key Points

- **Assumptions:** LDA assumes that:
 - Each class follows a Gaussian distribution with identical covariance matrices across all classes.
 - The data is normally distributed, and the classes have the same covariance.
 - LDA is sensitive to outliers and imbalances in the class distribution.
- LDA aims to maximize the distance between class means and minimize the variance within each class, thereby improving class separability.
- **Computationally Efficient:** LDA is generally faster and computationally simpler than non-linear dimensionality reduction methods like t-SNE or UMAP because it is based on linear transformations.
- **LDA vs. PCA:**
 - **PCA:** A data-driven, unsupervised technique that focuses on maximizing the total variance in the data, irrespective of class labels.
 - **LDA:** A supervised method that uses class labels to maximize the separability between classes, thereby focusing on finding dimensions that best distinguish between classes.

Singular Value Decomposition (SVD)

Singular Value Decomposition (SVD) is a powerful matrix factorization technique used in linear algebra, data analysis, and machine learning. It decomposes a matrix into three simpler matrices and is widely used for dimensionality reduction, noise reduction, and feature extraction.

Given a real matrix $A \in R^{m \times n}$, the SVD is defined as:

$$A = U\Sigma V^T$$

where:

- $U \in R^{m \times m}$ is an orthogonal matrix whose columns are the left singular vectors,
- $\Sigma \in R^{m \times n}$ is a diagonal matrix with non-negative real numbers on the diagonal (the singular values),
- $V \in R^{n \times n}$ is an orthogonal matrix whose columns are the right singular vectors.

Step-by-Step Dimensionality Reduction Using SVD

1. Perform SVD on A :

- Decompose the matrix A into its singular value decomposition:

$$A = U\Sigma V^T$$

- Here:
 - $U \in R^{m \times m}$ contains the left singular vectors as columns.
 - $\Sigma \in R^{m \times n}$ is a diagonal matrix with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, where $r = \text{rank}(A)$.
 - $V^T \in R^{n \times n}$ is the transpose of the matrix of right singular vectors.

2. Truncate U , Σ , and V^T to top- k components:

- To reduce the matrix A to a lower-dimensional approximation using the top k singular values:
 - Let $U_k \in R^{m \times k}$ be the first k columns of U ,
 - Let $\Sigma_k \in R^{k \times k}$ be the top-left $k \times k$ block of Σ ,
 - Let $V_k^T \in R^{k \times n}$ be the first k rows of V^T .
- The rank- k approximation of A is:

$$A_k = U_k \Sigma_k V_k^T$$

3. Compute the Reduced Representation A' :

- Often, a reduced feature representation is desired:

$$A' = AV_k = U_k \Sigma_k \in R^{m \times k}$$

- The matrix A' captures the essential structure of the original data in a lower-dimensional space of dimension k , preserving the most significant singular values and their directions.

Singular Value Decomposition (SVD) Calculation Steps

To compute SVD manually for a matrix $A \in R^{m \times n}$, follow the steps below:

1. Compute AA^T :

- Calculate the matrix product $AA^T \in R^{m \times m}$.
- Find its eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_m$ and the corresponding eigenvectors $u_1, u_2, \dots, u_m$.
- The eigenvectors form the columns of U , and the eigenvalues are used to compute singular values.

2. Compute $A^T A$:

- Compute $A^T A \in R^{n \times n}$.

- Find its eigenvalues and corresponding eigenvectors $v_1, v_2, \dots, v_n$.
- The eigenvectors form the columns of V .

3. Normalize the eigenvectors:

- Ensure that each eigenvector has unit length to construct orthogonal matrices U and V .

4. Sort eigenvalues and corresponding vectors:

- Sort the eigenvalues of AA^T and $A^T A$ in descending order.
- Reorder the corresponding eigenvectors to match the order of the sorted eigenvalues.

5. Compute Singular Values:

- The singular values σ_i are the square roots of the non-zero eigenvalues:

$$\sigma_i = \sqrt{\lambda_i}$$

6. Form the Diagonal Matrix Σ :

- Construct $\Sigma \in R^{m \times n}$ as:

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \\ \vdots & \vdots & & \vdots \end{bmatrix}$$

where $r = \text{rank}(A)$, and the rest of the matrix is padded with zeros.

7. Transpose V :

- The matrix V^T is simply the transpose of V .

8. Combine to Form the SVD:

- The original matrix A can now be written as:

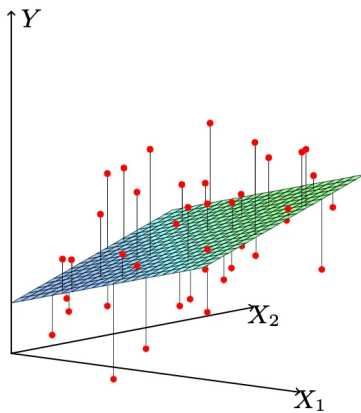
$$A = U\Sigma V^T$$

MACHINE LEARNING MODELS

Linear, Ridge, Lasso & Elastic Regression

Linear Regression

- X : Design matrix of input features with dimensions 1000×10 (including a column of ones for the intercept and 9 predictor variables).
- Y : Column vector of observed outcomes, with dimensions 1000×1 .
- β : Column vector of regression coefficients, with dimensions 10×1 .
- $\hat{Y}$: Column vector of predicted values, with dimensions 1000×1 .



Prediction

The predicted values $\hat{Y}$ are obtained using the linear model:

$$\hat{Y} = X\beta$$

1. Closed-Form Solution (Ordinary Least Squares)

We consider a linear regression model:

$$Y = X\beta + \epsilon$$

where:

- $Y \in \mathbb{R}^{n \times 1}$ is the response/output vector,
- $X \in \mathbb{R}^{n \times p}$ is the feature matrix (including a column of 1s for the intercept, if applicable),
- $\beta \in \mathbb{R}^{p \times 1}$ is the vector of coefficients to estimate,
- ϵ is the error term (noise).

Our goal is to find β that minimizes the **Residual Sum of Squares (RSS)**: $L(\beta) = \|Y - X\beta\|^2$

This is equivalent to minimizing the squared Euclidean norm of the residuals.

9. Step 1: Expand the loss function

Using the identity $\|a\|^2 = a^\top a$:

$$L(\beta) = (Y - X\beta)^\top (Y - X\beta)$$

Now expand using the distributive property of matrix multiplication:

$$\begin{aligned} L(\beta) &= Y^\top Y - Y^\top X\beta - \beta^\top X^\top Y + \beta^\top X^\top X\beta \\ &= Y^\top Y - 2\beta^\top X^\top Y + \beta^\top X^\top X\beta \end{aligned}$$

(Note: $Y^\top X\beta$ and $\beta^\top X^\top Y$ are scalars and equal; we combine them.)

10. Step 2: Take the derivative of $L(\beta)$ w.r.t. β

Using standard matrix calculus rules: $-\frac{\partial}{\partial \beta}(\beta^\top A\beta) = 2A\beta$ when A is symmetric, $-\frac{\partial}{\partial \beta}(b^\top \beta) = b$,

We get: $\frac{\partial L}{\partial \beta} = -2X^\top Y + 2X^\top X\beta$

11. Step 3: Set the derivative to zero for minimization

$$\begin{aligned} -2X^\top Y + 2X^\top X\beta &= 0 \\ X^\top X\beta &= X^\top Y \end{aligned}$$

12. Step 4: Solve for β

Assuming that $X^\top X$ is invertible:

$$\boxed{\beta = (X^\top X)^{-1} X^\top Y}$$

Note: If $X^\top X$ is not invertible (i.e., singular), we may use regularization (like Ridge Regression) or the Moore-Penrose pseudoinverse instead.

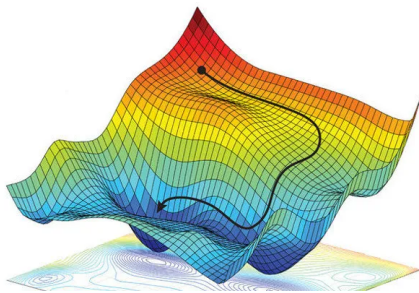
2. Iterative Solution (Gradient Descent)

An alternative approach is to use gradient descent to iteratively update the coefficients. For n epochs, the update rule is:

$$\nabla_\beta L = \frac{dL}{d\beta} = -2X^\top Y + 2X^\top X\beta$$

$$\beta^{(n)} = \beta^{(n-1)} - \frac{\alpha}{1000} \cdot \nabla_\beta L$$

where α is the learning rate and 1000 is the number of observations (used for averaging the gradient).



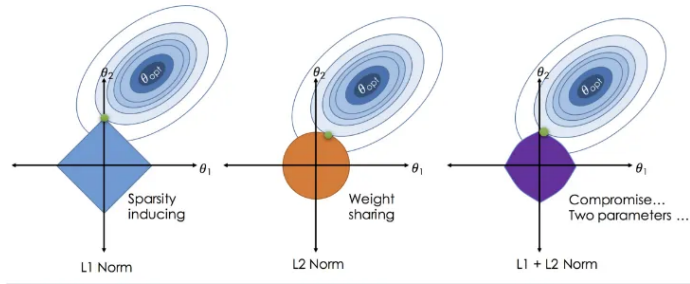
Polynomial Regression

To model non-linear relationships, polynomial features can be introduced. For example, given three input features x , y , and z , a second-degree polynomial expansion includes:

$$x^2, y^2, z^2, xy, xz, yz$$

These new features are added to the design matrix, and then standard linear regression (e.g., OLS or gradient descent) is applied on the transformed feature space.

Ridge, Lasso and Elastic Net Regression



Ridge Regression (L2 Regularization)

Ridge regression is a regularized version of linear regression that adds an L2 penalty to the loss function, thereby discouraging large coefficients and reducing model complexity:

$$L(\beta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n \beta_j^2$$

Where:

- $L(\beta)$ is the loss function, comprising the residual sum of squares and the regularization term.
- λ is the regularization parameter that controls the magnitude of the penalty.

The gradient of the loss function with respect to the coefficients is given by:

$$\frac{\partial L}{\partial \beta} = -2X^\top(Y - X\beta) + 2\lambda X^\top X\beta$$

Here, $X^\top X$ is the Gram matrix of the input features, and λI is the regularization matrix. The identity matrix I is given by:

$$I = \begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

The closed-form solution for the Ridge regression coefficients is:

$$\beta = (X^\top X + \lambda I)^{-1} X^\top Y$$

Alternatively, using gradient descent, the update rule for the coefficients is:

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \frac{\partial L}{\partial \beta^{(t)}}$$

Where α is the learning rate and $\beta^{(t)}$ represents the coefficient vector at the t -th iteration.

For a simple linear regression model, the slope m under Ridge regularization is computed as:

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x})}{\sum (x_i - \bar{x})^2 + \lambda}$$

Note: As λ increases, the magnitude of the coefficients decreases, but they never reach zero. Therefore, Ridge regression is not typically used for feature selection.

Lasso Regression (L1 Regularization)

Lasso regression introduces an L1 penalty to the loss function, which encourages sparsity in the coefficient estimates, effectively performing feature selection:

$$L(\beta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |\beta_j|$$

For the slope m in the case of simple linear regression, Lasso regression modifies the computation as follows:

- For $m > 0$:

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x}) - \lambda}{\sum (x_i - \bar{x})^2}$$

- For $m = 0$:

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x})}{\sum (x_i - \bar{x})^2}$$

- For $m < 0$:

$$m = \frac{\sum (y_i - \bar{y})(x_i - \bar{x}) + \lambda}{\sum (x_i - \bar{x})^2}$$

Lasso regression is particularly effective when you wish to eliminate some features entirely by setting their coefficients to zero, especially when λ is large.

Note: Lasso is often used for feature selection because, as λ increases, it forces some coefficients to become exactly zero, thereby removing those features from the model.

Elastic Net Regression

Elastic Net regression combines both L1 (Lasso) and L2 (Ridge) regularization penalties, offering a balance between the two methods. This is useful when there are multiple correlated features or when the number of predictors exceeds the number of observations.

The Elastic Net loss function is:

$$L(\beta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^n |\beta_j| + \lambda_2 \sum_{j=1}^n \beta_j^2$$

Where:

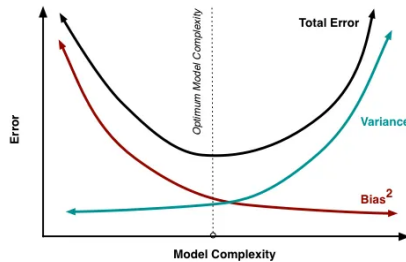
- λ_1 controls the strength of the L1 penalty (Lasso).
- λ_2 controls the strength of the L2 penalty (Ridge).
- When $\lambda_1 = 0$ and $\lambda_2 > 0$, the model reduces to Ridge regression.
- When $\lambda_2 = 0$ and $\lambda_1 > 0$, the model reduces to Lasso regression.
- Elastic Net is effective when there are many correlated predictors, as it encourages grouped selection of correlated features.

Properties of Ridge and Lasso Regression

(a) Effect of increasing λ on coefficients:

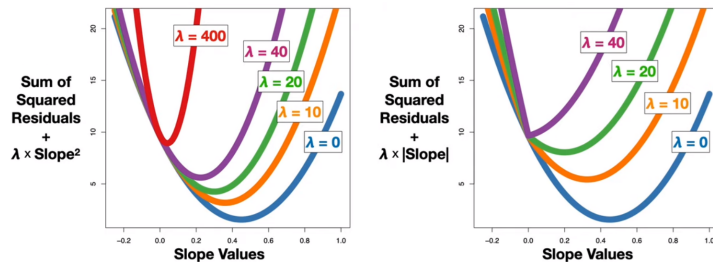
- **Ridge Regression:** As λ increases, the coefficients decrease toward zero but never exactly reach zero. This leads to a reduction in the magnitude of the coefficients, but they are never completely eliminated. Thus, for Ridge regression, $m \approx 0$, but $m \neq 0$.
- **Lasso Regression:** As λ increases, the coefficients can shrink to exactly zero, which leads to feature selection. Therefore, for Lasso regression, $m = 0$ for some coefficients.

(b) Bias-Variance Tradeoff:



(c) Impact on Loss Function:

- As λ increases, the minimum of the loss function increases. This reflects the fact that we are penalizing the coefficients more heavily, leading to a tradeoff between fitting the data and keeping the coefficients small.
- As λ increases, the coefficients (m) get closer to zero, thus reducing the model's complexity.



Logistic & Multinomial Logistic Regression

Logistic Regression

1. Perceptron Trick (Motivation)

The perceptron algorithm is an early classification method that updates weights when it misclassifies data. The idea is to nudge the decision boundary to better separate classes.

- If a point is misclassified, we move the decision boundary toward it.
- **Limitation:** The perceptron stops once it finds a separating hyperplane, not necessarily the best one.

Explanation: The perceptron algorithm updates its weights only when a prediction is wrong, encouraging the model to correct its decision boundary. However, it doesn't give us probabilistic predictions, and it only works well when the data is linearly separable. This leads us to the need for a more refined model—Logistic Regression.

Algorithm 1 Perceptron Learning

Require: Input data $X \in R^{m \times n}$, labels $Y \in \{\pm 1\}^m$, learning rate η

Ensure: Trained weight vector W

```
1: Initialize  $W$  randomly
2: for  $t = 1$  to 1000 do
3:   Select random index  $i$ 
4:   Predict:  $\hat{y}_i = \text{sign}(x_i^T W)$ 
5:   Update:  $W = W + \eta(y_i - \hat{y}_i)x_i$ 
6: end for
```

Notation:

- x_i : i^{th} input row vector
- W : weight vector ($n \times 1$)
- y_i : true label, $\hat{y}_i$: predicted label
- η : learning rate

2. Sigmoid Function and Stochastic Logistic Regression

The perceptron is replaced by the sigmoid function for probabilistic interpretation and smooth optimization:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function maps any real number into the range (0, 1). It's the core of logistic regression.

Explanation: Logistic regression uses the sigmoid function to estimate the probability that a sample belongs to a particular class. Unlike the hard decision rule in perceptron, the sigmoid gives smooth output, making it differentiable and suitable for optimization using gradient-based methods.

Algorithm 2 Stochastic Logistic Regression

Require: $X \in R^{m \times n}$, $Y \in \{0, 1\}^m$, learning rate η

Ensure: Weights W

```
1: Initialize  $W$  with small values
2: for iteration = 1 to 1000 do
3:   Randomly pick  $(x_i, y_i)$ 
4:   Predict:  $\hat{y}_i = \sigma(x_i^T W)$ 
5:   Update:  $W = W + \eta(y_i - \hat{y}_i)x_i$ 
6: end for
```

3. Maximum Likelihood & Batch Training

Logistic regression fits weights using maximum likelihood estimation, which minimizes the following binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Explanation: Instead of updating weights one sample at a time, we can use all training samples together in batch mode. This is done by minimizing the log-loss, also known as binary cross-entropy. The loss penalizes incorrect predictions more heavily as the confidence increases in the wrong direction.

Matrix Formulation

$$\hat{Y} = \sigma(XW)$$

$$\mathcal{L} = -\frac{1}{m} [Y^T \log(\hat{Y}) + (1 - Y)^T \log(1 - \hat{Y})]$$

Explanation: In matrix form, the loss function and prediction become compact and computationally efficient. This helps implement batch gradient descent easily using matrix operations.

Gradient Descent Update Rule

$$W \leftarrow W + \eta \cdot \frac{1}{m} X^T (Y - \hat{Y})$$

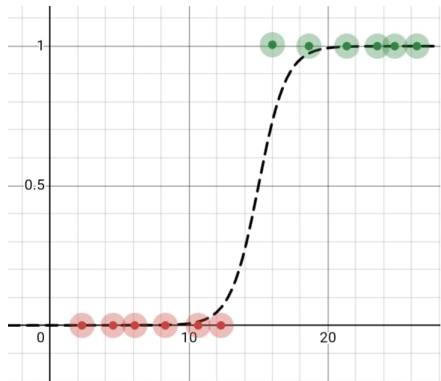
Explanation: The gradient is calculated from the derivative of the loss with respect to the weights. This gradient points in the direction to increase likelihood (or decrease loss), and we move the weights slightly in this direction.

Algorithm 3 Batch Gradient Descent for Logistic Regression

Require: Input matrix X , label vector Y , learning rate η

Ensure: Trained weight vector W

- 1: Initialize W with small values
 - 2: **for** epoch = 1 to 10 **do**
 - 3: Predict: $\hat{Y} = \sigma(XW)$
 - 4: Compute gradient: $\nabla = \frac{1}{m} X^T (Y - \hat{Y})$
 - 5: Update: $W \leftarrow W + \eta \cdot \nabla$
 - 6: **end for**
-



Softmax Regression / Multinomial Logistic Regression

Softmax regression generalizes logistic regression to multi-class classification problems. Instead of modeling a binary outcome, it predicts the probability distribution over K classes.

Method One: One-vs-Rest Approach (Simplified View)

- Apply one-hot encoding to convert categorical labels into a binary matrix format.

Feature (Color)		Red	Green	Yellow
Red	One Hot Encoding →	1	0	0
Green		0	1	0
Yellow		0	0	1
Green		0	1	0
Red		1	0	0

- Train a **separate binary logistic regression model for each class** (i.e., one model per column of the one-hot encoded labels).
- Each model predicts the probability of its respective class vs all others.
- The class with the highest predicted probability is chosen during inference.

Method Two: Softmax-Based Training (True Multinomial Logistic Regression)

In this method, we train a **single model** that jointly predicts all classes using the softmax function. This avoids the independence assumption of Method One.

Loss Function (Cross-Entropy for Multiple Classes)

$$\text{Loss} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left(y_k^{(i)} \cdot \log(\hat{y}_k^{(i)}) \right)$$

- Here, $y_k^{(i)} = 1$ if the i^{th} example belongs to class k , and 0 otherwise (from one-hot encoding).
- $\hat{y}_k^{(i)}$ is the predicted probability for class k for the i^{th} example.

Softmax Function

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K$$

- Converts raw scores (logits) into probabilities.
- Ensures output values are between 0 and 1 and sum to 1 across all classes.

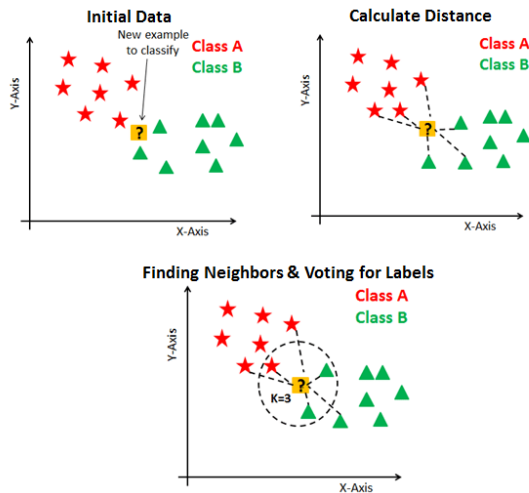
Important Points

- Commonly used for multi-class classification tasks (e.g., digit recognition, sentiment with 3+ categories).
- We can apply regularization to prevent overfitting:
 - **L1 Regularization (Lasso):** Encourages sparsity in weights.
 - **L2 Regularization (Ridge):** Encourages smaller weights overall.
 - **Elastic Net:** A mix of both L1 and L2.
- Optimization is usually done via batch or stochastic gradient descent.

K-Nearest Neighbors (KNN)

Based on the concept: “You are the average of the five people you spend the most time with”

K-Nearest Neighbors (KNN) is a non-parametric, instance-based learning algorithm used for both classification and regression tasks. It predicts outcomes based on the similarity of data points in the feature space.



Algorithm Steps

Step 1: Normalize the Data

- Scale features appropriately (e.g., min-max normalization):

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

Step 2: Calculate Distances

- Euclidean distance:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- Other options: Manhattan, Minkowski, or Cosine similarity

Step 3: Identify K-Nearest Neighbors

- Select top K points with smallest distances
- Optimal K is typically chosen via cross-validation

Step 4: Make Prediction

- **Classification:** Majority vote

$$\hat{y} = (\{y_i\}_{i=1}^K)$$

- **Regression:** Weighted average

$$\hat{y} = \frac{\sum_{i=1}^K w_i y_i}{\sum_{i=1}^K w_i}, \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)}$$

Naive Bayes Classifier

Bayes' Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Classification Formulation

For features A, B, C and class C_k :

$$P(C_k|A, B, C) = \frac{P(A|C_k)P(B|C_k)P(C|C_k)P(C_k)}{P(A, B, C)}$$

Using the Naive assumption (feature independence):

$$\hat{y} = \arg \max_{C_k} P(C_k) \prod_{i=1}^n P(x_i|C_k)$$

Example Calculation

Toss	Venue	Outlook	Result
Won	Mumbai	Overcast	Won
Lost	Chennai	Sunny	Won
$\vdots$	$\vdots$	$\vdots$	$\vdots$
Won	Mumbai	Sunny	Won

For $P(\text{Won}|\text{Lost}, \text{Mumbai}, \text{Sunny})$:

$$\propto P(\text{Lost}|\text{Won}) \cdot P(\text{Mumbai}|\text{Won}) \cdot P(\text{Sunny}|\text{Won}) \cdot P(\text{Won}) = \frac{1}{5} \times \frac{2}{5} \times \frac{4}{5} \times \frac{5}{9} \approx 0.0356$$

Numerical Feature Handling

For continuous data, Gaussian Naive Bayes assumes the likelihood of the features is Gaussian:

$$P(x|C_k) = \frac{1}{\sqrt{2\pi\sigma_{C_k}^2}} \exp\left(-\frac{(x - \mu_{C_k})^2}{2\sigma_{C_k}^2}\right)$$

Implementation Steps

- Calculate the mean μ_{C_k} and variance $\sigma_{C_k}^2$ of each feature for every class.
- Use the Gaussian formula to compute the likelihood for each feature.
- Multiply all feature likelihoods with the class prior to get posterior probabilities.
- Choose the class with the highest posterior probability.

CART - Classification and Regression Trees

Pseudo Code

- (a) Begin with the training dataset containing feature variables and corresponding target values.
- (b) Determine the **best feature** to split the data on using a criterion like Gini impurity or entropy.
- (c) Split the dataset into subsets based on this feature. Each split defines a node in the decision tree.
- (d) Recursively repeat steps 2 and 3 for each subset until a stopping criterion is met (e.g., maximum depth, pure leaf, or minimum samples).

Advantages

- Fast inference: prediction cost is logarithmic in the number of training samples.
- Easy to interpret and visualize.

Disadvantages

- Prone to **overfitting**, especially with deep trees.
- Sensitive to imbalanced datasets and small variations in data.

Decision Tree Concepts

Entropy (Information Gain)

$$\text{Entropy}(X) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where p_i is the proportion of samples belonging to class i .

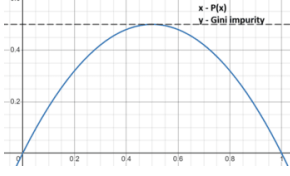
- For a binary classification problem:
 - Minimum Entropy = 0 (pure node)
 - Maximum Entropy = 1 (uniform distribution)
- For multiclass problems, the maximum entropy > 1 .
- Both $\log_2$ and $\log_e$ (natural log) are valid; base 2 is commonly used in decision trees.



Gini Impurity

$$G = 1 - \sum_{i=1}^c p_i^2$$

- Gini tends to give balanced trees in comparison to entropy.
- Lower Gini implies purer node.



Information Gain

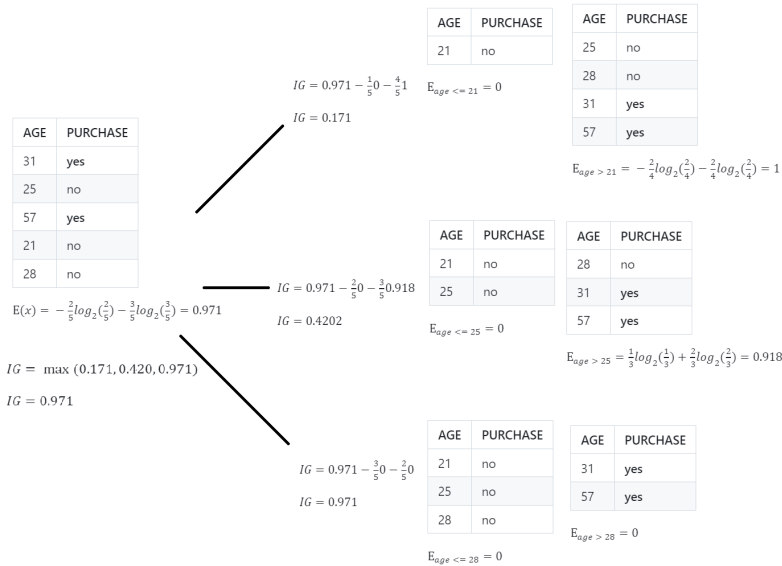
$$\begin{aligned} \text{Information Gain} &= \text{Impurity (Parent)} - \sum_{i=1}^k \frac{w_i}{W} \cdot \text{Impurity (Child}_i) \\ &= \text{Entropy or Gini(Parent)} - \text{Weighted Average of Child Impurities} \end{aligned}$$

Algorithm

- Compute impurity (Entropy or Gini) of the parent node.
- For each feature:
 - Split the dataset based on feature values.
 - Compute impurity of child nodes and their weighted average.
 - Calculate the information gain.
- Select the feature with the highest information gain to split.
- Stop when:
 - All samples belong to one class (pure node, entropy = 0), or
 - Maximum depth or minimum node size is reached.

Calculating Information Gain for Numerical Features

- Sort the dataset based on the values of the numerical feature.
- Consider potential split points between each pair of consecutive values.
- For each split point:
 - Divide the dataset into two subsets: values less than or equal to the split, and values greater.
 - Compute the Information Gain using Entropy or Gini impurity.
- Select the split point that yields the maximum Information Gain.
- The corresponding Information Gain is assigned to that numerical feature.



Regression Trees

Algorithm

- Compute the standard deviation of the target variable in the parent node.
- For each feature (categorical or numerical), calculate the Information Gain:
 - **Categorical Features:**
 - * Split the dataset based on the unique values (labels) of the categorical feature.
 - * Compute the standard deviation of the target variable for each group (child node).
 - **Numerical Features:**
 - * Sort the data based on the numerical feature.
 - * Consider possible split points between consecutive values.
 - * For each split, divide the dataset into two groups.
 - * Compute the standard deviation of the target variable in each group (child node).
 - Compute the weighted standard deviation of the child nodes.
 - Calculate Information Gain as:

$$\text{Information Gain} = \text{Standard Deviation (Parent)} - \text{Weighted Standard Deviation (Children)}$$

- Select the feature and split that results in the highest Information Gain.
- Recursively repeat the above steps for each child node until a stopping criterion is met (e.g., minimum samples per node or maximum tree depth).
- At each leaf node, output the mean of the target variable values within that node.

Feature Importance for Decision Tree-like Algorithms

To compute the importance of a feature i , sum the decrease in impurity for all nodes t where the feature i is used to split:

$$\text{Feature Importance}(i) = \sum_{t \in \text{nodes using } i} \frac{N_t}{N} \left(\text{impurity}_t - \frac{N_{t_l}}{N_t} \cdot \text{impurity}_{t_l} - \frac{N_{t_r}}{N_t} \cdot \text{impurity}_{t_r} \right)$$

- $\text{Feature Importance}(i)$: Importance score for feature i .
- N_t : Number of samples at node t .
- N : Total number of samples in the dataset.
- impurity_t : Impurity at node t (e.g., Gini, Entropy, Standard Deviation).
- N_{t_l} : Number of samples in the left child node.
- N_{t_r} : Number of samples in the right child node.
- impurity_{t_l} : Impurity of the left child node.
- impurity_{t_r} : Impurity of the right child node.

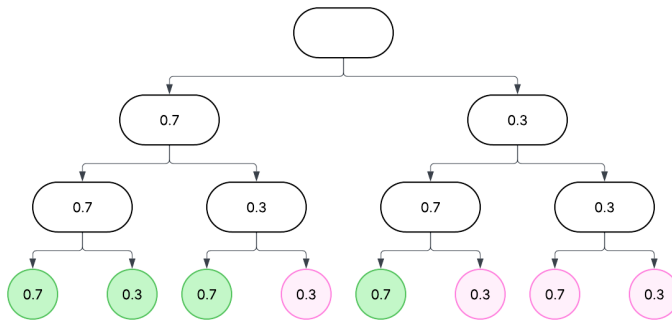
Ensemble Learning

Ensemble learning is based on the concept of the *wisdom of the crowd*, where combining predictions from multiple models increases the overall accuracy and robustness of the system. It helps convert a low-bias, high-variance model into a low-bias, low-variance model by distributing the impact of random errors and outliers across multiple learners.

- **Voting Ensemble:** In this method, multiple diverse models (e.g., SVM, Decision Tree, Logistic Regression) make predictions independently.
 - For **classification**, the final output is determined by majority voting or by averaging class probabilities (soft voting).
 - For **regression**, the mean of the individual model predictions is used as the final result.
- **Stacking:** Stacking combines different models by using their predictions as input features for a final meta-model (e.g., KNN or Logistic Regression).
 - Base learners (e.g., SVM, Decision Tree, Logistic Regression) are first trained on the dataset.
 - Their predictions are then used as input to train a meta-model that learns to optimally combine these outputs.
- **Bagging (Bootstrap Aggregating):** Bagging involves training multiple instances of the same model on different random subsets (with replacement) of the training data.
 - The final prediction is obtained through majority voting (classification) or averaging (regression).
 - Bagging helps reduce variance and is effective for high-variance, low-bias models like decision trees.
- **Boosting:** Boosting is a sequential ensemble technique where each model tries to correct the errors of its predecessor.
 - Models are trained one after the other.
 - Each subsequent model focuses more on the data points that were misclassified by the previous models.
 - The final prediction is typically a weighted sum of individual model outputs.

Voting Ensemble Method

Assume we have three independent models, each with an individual accuracy of 0.7.



Ensemble Accuracy Calculation (Majority Voting):

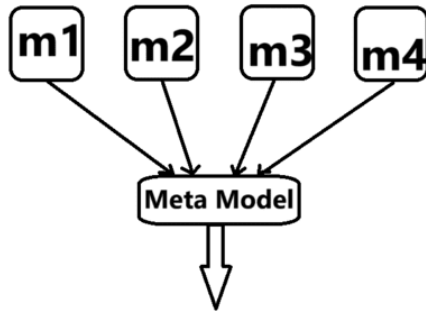
We calculate the probability that the ensemble gives the correct result when at least 2 out of 3 models are correct:

$$\text{Final Accuracy} = (0.7)^3 + 3 \times (0.7)^2 \times 0.3 = 0.343 + 0.441 = \boxed{0.784}$$

Types of Voting:

- **Hard Voting:** The class predicted by the majority of the models (i.e., mode of predictions) is selected.
- **Soft Voting:** The class with the highest average predicted probability (i.e., mean of probabilities) is selected.

Stacking



1. Hold-Out Method (Blending)

The hold-out method, also known as **blending**, is a basic form of stacking. In this method, the training dataset is split into two parts:

- **Base Model Training:** The first part is used to train the base models.
- **Meta-Model Training:** The trained base models make predictions on the second part of the data. These predictions serve as input features to the meta-model (often a simple model like linear or logistic regression).
- The meta-model is then trained on these predictions to learn how to best combine the base model outputs for final prediction.

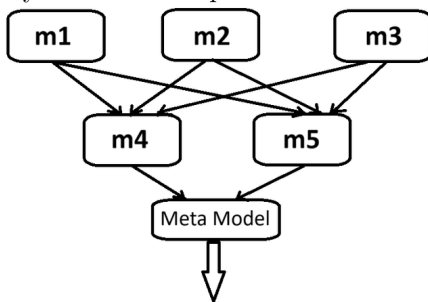
2. K-Fold Approach (Stacking)

The **K-Fold stacking** method improves upon blending by using cross-validation to better utilize the training data and reduce overfitting:

- **Data Splitting:** Divide the training data into K folds.
- **Base Model Training:** For each base model type, train K different models. In each iteration, use $K-1$ folds for training and the remaining fold for validation. This ensures each data point is used for validation once.
- **Out-of-Fold Predictions:** Collect the out-of-fold predictions made by each of the K trained models. These predictions form the new feature set for the meta-model.
- **Meta-Model Training:** Train the meta-model on the out-of-fold predictions from all base models.
- **Final Model Retraining:** After training the meta-model, retrain each base model type on the entire training dataset to be used during final inference.

3. Multi-Layered Stacking

Multi-layered stacking extends the idea by creating a hierarchy of models across multiple layers. The output of each layer is used as input for the next:



This approach enables models to learn from previous model predictions, enhancing overall performance—especially when using a diverse set of model types.

Bagging Techniques

Bagging (Bootstrap Aggregating) is an ensemble method that improves model stability and accuracy by training multiple models on different subsets of the data and combining their predictions. Below are some of the common techniques used in bagging:

- **Row Sampling with Replacement:** This is the classic bagging technique where each individual model is trained on a bootstrap sample, meaning data points are selected randomly with replacement. Some data points may appear multiple times in the sample, while others may not be included at all.
- **Pasting:** Similar to row sampling, but in this case, data is sampled without replacement. Each model is trained on a unique subset of the data, ensuring that every data point is used exactly once.
- **Random Subspaces:** This method focuses on column sampling (i.e., selecting a subset of features). Models are trained on different subsets of features, either with or without replacement, allowing the ensemble to capture different aspects of the data.
- **Random Patches:** A combination of both row and column sampling. In this technique, both subsets of rows (data points) and columns (features) are selected, with or without replacement. This enables the ensemble to capture diverse patterns from both the data and the feature set.

Out-of-Bag (OOB) Error: In bagging, approximately 37% of the samples are not selected for training any given model. These samples, called out-of-bag (OOB) samples, can be used as a validation set to evaluate the model's performance. This internal validation provides a cost-effective way of assessing the model's accuracy without the need for a separate test set.

AdaBoost Algorithm

1. Initialize Weights

For a dataset with n samples, initialize the weight for each sample equally:

$$w_i = \frac{1}{n}, \quad \text{for all } i = 1, 2, \dots, n$$

This ensures that all samples have equal influence initially.

x	y	w_i
		$\frac{1}{n}$

2. Train a Weak Learner

Train a weak learner (typically a decision stump, i.e., a tree of depth 1) on the training data using the current sample weights. A weak learner is expected to perform only slightly better than random guessing.

3. Make Predictions

Use the trained weak learner to predict the labels for all training samples. Denote these predictions as $\hat{y}_i$.

x	y	w_i	$\hat{y}$
		$\frac{1}{n}$	

4. Compute Weighted Error

Calculate the weighted error ϵ of the weak learner:

$$\epsilon = \sum_{i=1}^n w_i \cdot I(\hat{y}_i \neq y_i)$$

where $I(\cdot)$ is the indicator function that equals 1 when the prediction is incorrect and 0 otherwise.

x	y	w_i	$\hat{y}$	ϵ
		$\frac{1}{n}$		

5. Compute Learner Weight (α)

Calculate the weight α for the weak learner, which reflects its contribution to the final model:

$$\alpha = \frac{1}{2} \log \left(\frac{1 - \epsilon}{\epsilon} \right)$$

A lower error ϵ results in a higher α , indicating a more reliable learner.

6. Update Sample Weights

Update the weights of each sample depending on whether it was correctly or incorrectly classified:

$$w_i^{\text{new}} = w_i \cdot e^{-\alpha y_i \hat{y}_i}$$

This formulation increases weights for misclassified samples and decreases them for correctly classified ones.

x	y	w_i	$\hat{y}$	ϵ	w_i^{new}
		$\frac{1}{n}$			

7. Normalize Weights

Normalize the updated weights so that they sum to 1:

$$w_i^{\text{norm}} = \frac{w_i^{\text{new}}}{\sum_{j=1}^n w_j^{\text{new}}}$$

x	y	w_i	$\hat{y}$	ϵ	w_i^{new}	w_i^{norm}
		$\frac{1}{n}$				

8. Optional: Create Sampling Bins

You may optionally build bins for sampling based on the normalized weights. Each sample is assigned a probability interval:

$$\text{bin}_i = (a_i, b_i) = (b_{i-1}, b_{i-1} + w_i^{\text{norm}})$$

x	y	w_i	$\hat{y}$	ϵ	w_i^{new}	w_i^{norm}	$\text{bin}_i = (a_i, b_i)$
		$\frac{1}{n}$					

9. Repeat

Repeat steps 2 through 7 for a specified number of iterations or until convergence. **Note:** Do not reinitialize weights to $\frac{1}{n}$ in each iteration; the updated weights are carried forward to the next round.

10. Final Prediction

After T rounds, the final prediction is a weighted majority vote (or sum) of all weak learners:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$$

where:

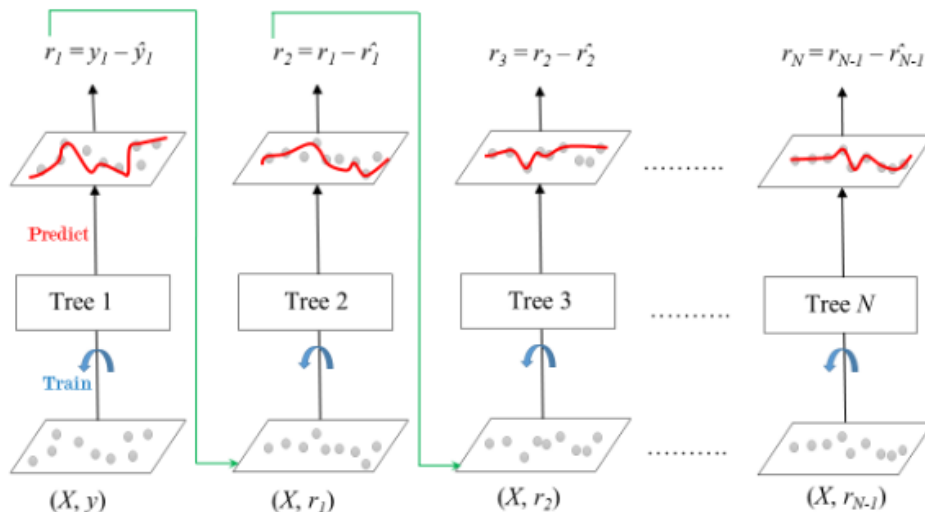
- $h_t(x)$ is the prediction of the t -th weak learner,
- α_t is the weight of the t -th weak learner,
- $H(x)$ is the final ensemble prediction.

Summary: AdaBoost sequentially trains weak learners on reweighted data, focusing more on previously misclassified samples. It then combines them into a strong learner using a weighted vote, significantly boosting performance.

Gradient Boosting

The final prediction is a weighted sum of all models:

$$\text{Result} = \text{Model}_0(X) + \eta \cdot \text{Model}_1(X) + \eta \cdot \text{Model}_2(X) + \cdots + \eta \cdot \text{Model}_n(X)$$



Gradient Boosting for Regression

Algorithm 4 Gradient Boosting for Regression

- 1: **Input:** Training data (X, y) , number of estimators n , learning rate η
- 2: Initialize model with constant prediction:

$$F_0(x) = \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

- 3: $\text{modelsList} \leftarrow [F_0]$
- 4: $F(x) \leftarrow F_0(x)$
- 5: **for** $i \leftarrow 1$ to n **do**
- 6: Compute residuals:

$$r_i = y_i - F(x_i), \quad \text{for all } x_i \in X$$

- 7: Fit regression tree $h_i(x)$ to predict r_i from x_i
- 8: Update model:

$$F(x) \leftarrow F(x) + \eta \cdot h_i(x)$$

- 9: $\text{modelsList.append}(h_i)$
- 10: **end for**
- 11: **Output:** Final prediction function:

$$\hat{y}(x) = F_0(x) + \sum_{i=1}^n \eta \cdot h_i(x)$$

Gradient Boosting for Classification

Algorithm 5 Gradient Boosting for Binary Classification

- 1: **Input:** Training data (X, y) , learning rate η , number of estimators n
- 2: Compute initial log-odds:

$$F_0 = \ln \left(\frac{\sum y}{\sum (1 - y)} \right)$$

- 3: $\text{modelsList} \leftarrow [F_0]$
- 4: **for** $i \leftarrow 1$ to n **do**
- 5: Compute predicted probabilities:

$$\hat{p}_k = \frac{1}{1 + e^{-F_{i-1}(x_k)}}, \quad \text{for all } x_k \in X$$

- 6: Compute residuals (negative gradient):

$$r_k = y_k - \hat{p}_k, \quad \text{for all } x_k \in X$$

- 7: Fit a regression tree $h_i(x)$ to predict r_k from x_k
- 8: **for** each leaf j in h_i with region R_j **do**
- 9: Compute leaf output:

$$\gamma_j = \frac{\sum_{x_k \in R_j} r_k}{\sum_{x_k \in R_j} \hat{p}_k (1 - \hat{p}_k)}$$

- 10: Assign γ_j to all $x_k \in R_j$
- 11: **end for**
- 12: Update prediction:

$$F_i(x) = F_{i-1}(x) + \eta \cdot h_i(x)$$

- 13: $\text{modelsList.append}(h_i)$
- 14: **end for**
- 15: **Output:** Final prediction:

$$F(x) = F_0 + \sum_{i=1}^n \eta \cdot h_i(x)$$

$$\hat{p}(x) = \frac{1}{1 + e^{-F(x)}}, \quad \hat{y} = \begin{cases} 1 & \text{if } \hat{p}(x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Gradient Boosting Algorithm (General Formulation)

Given:

- Dataset: $\{(x_i, y_i)\}_{i=1}^n$
- Differentiable loss function: $L(y, F(x))$ (e.g., squared error: $L(y, F(x)) = \frac{1}{2}(y - F(x))^2$)
- Number of boosting iterations: M
- Learning rate: η

Goal: Build an additive model $F_M(x)$ minimizing the empirical risk:

$$F_M(x) = \sum_{m=0}^M \eta \cdot h_m(x), \quad \text{where } h_m(x) \text{ are weak learners}$$

Algorithm Steps

Algorithm 6 Gradient Boosting (General Form)

1: Initialize model:

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$$

▷ e.g., for squared loss: $F_0(x) = \frac{1}{n} \sum_{i=1}^n y_i$

2: **for** $m = 1$ to M **do**

3: Compute pseudo-residuals:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

▷ For squared loss: $r_{im} = y_i - F_{m-1}(x_i)$

4: Fit a regression tree $h_m(x)$ to training data $\{(x_i, r_{im})\}_{i=1}^n$

5: **for** each terminal region R_{jm} of h_m **do**

6: Compute region-specific update:

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$

▷ For squared loss: $\gamma_{jm} = \text{mean}\{r_{im} \mid x_i \in R_{jm}\}$

7: **end for**

8: Update model:

$$F_m(x) = F_{m-1}(x) + \eta \sum_{j=1}^{J_m} \gamma_{jm} \cdot \mathbf{1}_{\{x \in R_{jm}\}}$$

9: **end for**

10: **Output:** Final model $F_M(x)$

Key Components

- **Gradient Computation:** Uses first-order Taylor approximation (gradient descent in function space)
- **Weak Learner:** Usually regression trees with limited depth (e.g., depth 3–6)
- **Learning Rate η :** Shrinks contribution of each tree; common values: 0.01 to 0.1
- **Regularization:** Achieved by early stopping, limiting tree depth, subsampling, and η

XGBoost (Extreme Gradient Boosting)

Key Advantages of XGBoost

- **Flexibility**
 - **Cross-Platform Support:** Runs on Windows, Linux, and macOS
 - **Multi-Language APIs:** Available in Python, R, Java, Scala, and C++
 - **Integration Support:** Compatible with scikit-learn, Dask, Spark
 - **Wide Application:** Supports regression, classification, time series, and ranking tasks
- **Speed** (Often 10× faster than traditional gradient boosting)
 - **Parallelized Tree Construction:** Controlled via `n_jobs` (in scikit-learn API) or `nthread` (in core XGBoost)
 - **Columnar Data Storage:** Enables cache-aware access patterns
 - **Cache Optimization:** Minimizes memory overhead
 - **Out-of-Core Learning:** Handles datasets larger than memory using `tree_method = hist`
 - **Distributed Training:** Supports large-scale training across clusters
 - **GPU Acceleration:** Leverages GPUs via `device = cuda`
- **Performance**
 - **Regularization:** Incorporates both L1 (Lasso) and L2 (Ridge) penalties
 - **Robust to Missing Data:** Automatically learns optimal default directions
 - **Sparse Data Handling:** Efficient processing of sparse input matrices
 - **Efficient Split Finding:**
 - * Uses weighted quantile sketch for approximate split finding
 - * Employs binned feature values to speed up computation
 - **Tree Pruning:** Performs depth-first expansion with backward pruning to eliminate splits with negative gain

Technical Highlights (Simplified and Expanded)

- **Regularized Learning Objective:**

$$\mathcal{L}(\phi) = \sum_{i=1}^n l(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k), \quad \text{where } \Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

- The first term measures how well the model fits the data using a loss function $l(\cdot)$ (e.g., squared error for regression, log loss for classification).
- The second term $\Omega(f_k)$ adds a penalty for model complexity:
 - * T is the number of leaves (terminal nodes) in the tree.
 - * w represents the vector of weights (predictions) in each leaf.
 - * γ and λ are regularization parameters that help prevent overfitting.
- **Approximate Learning Algorithm (Histogram-based):**
 - Instead of evaluating every possible split, XGBoost uses an approximate method for faster computation.
 - Continuous features are grouped into discrete bins using quantiles (this forms a histogram).
 - For each bin, it calculates the sum of gradients and second-order gradients (Hessians).
 - This significantly speeds up the split finding process, especially on large datasets.
 - The method is called `tree_method = hist`, and is suitable for both CPU and GPU training.
- **Sparsity-Aware Split Finding:**
 - XGBoost is optimized to handle datasets with missing or sparse values.
 - During training, it automatically learns the best way to assign missing values (i.e., go left or right in a tree node).
 - This avoids the need for manual imputation and improves performance on real-world datasets with incomplete entries.
 - It also efficiently handles features with many zero entries (common in one-hot encoded or bag-of-words data).

XGBoost for Regression

Algorithm 7 XGBoost Regression Algorithm

1: **Initialize:**

prediction $\leftarrow$ mean(y)

models_list $\leftarrow$ [mean(y)]

2: **for** $i \leftarrow 1$ to n_estimators **do**

3: **Calculate residuals:** residual $\leftarrow y - \text{prediction}$

4: **Build decision tree:**

5: **for** each possible split **do**

6: Compute **Similarity Score:** $SS = \frac{(\sum \text{residuals})^2}{\text{count}(\text{residuals}) + \lambda}$

7: Compute **Gain:** Gain = $SS_{\text{left}} + SS_{\text{right}} - SS_{\text{parent}}$

8: Select split with maximum Gain

9: **end for**

10: Set **node output:** output = $\frac{\sum \text{residuals}}{\text{count}(\text{residuals}) + \lambda}$

11: **Update prediction:** prediction $\leftarrow$ prediction + $\eta \cdot \text{tree.predict}(X)$

12: models_list.append(tree)

13: **end for**

14: **Final prediction:**

$$\text{result} = \text{models_list}[0] + \eta \cdot \sum_{i=1}^{\text{n_estimators}} \text{models_list}[i](X)$$

XGBoost for Classification

Algorithm 8 XGBoost Classification Algorithm

1: **Initialize log odds:**

$$\text{log_odds} \leftarrow \ln \left(\frac{\text{Count of Ones}}{\text{Count of Zeros}} \right)$$

modelsList $\leftarrow$ [log_odds]

2: **for** $i \leftarrow 1$ to $n_{\text{estimators}}$ **do**

3: **Calculate probability:** prob $\leftarrow \frac{1}{1 + e^{-\text{log_odds}}}$

4: **Compute residuals:** residual $\leftarrow y - \text{prob}$

5: **Build decision tree:**

6: **for** each possible split **do**

7: Compute **Similarity Score:** $SS = \frac{(\sum \text{residuals})^2}{\sum [\text{prob} \times (1 - \text{prob})] + \lambda}$

8: Compute **Gain:** Gain = $SS_{\text{left}} + SS_{\text{right}} - SS_{\text{parent}}$

9: Select split with maximum Gain

10: **end for**

11: Set **node output:** log_loss = $\frac{\sum \text{residuals}}{\sum [\text{prob} \times (1 - \text{prob})] + \lambda}$

12: modelsList.append(tree)

13: **Update log odds:** log_odds $\leftarrow$ log_odds + $\eta \cdot \text{log_loss}$

14: **end for**

15: **Final prediction:**

$$\text{Total_log_loss} = \text{modelsList}[0] + \eta \cdot \sum_{i=1}^{n_{\text{estimators}}} \text{modelsList}[i](X)$$

$$\text{Final_prob} = \frac{1}{1 + e^{-\text{Total_log_loss}}}$$

XGBoost Mathematics

1. Objective Function

XGBoost minimizes the following regularized objective function at each boosting iteration t :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n L\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

- L : loss function (e.g., squared error or logistic loss)
- $\hat{y}_i^{(t-1)}$: prediction from previous iterations
- f_t : the function (tree) added at iteration t
- $\Omega(f_t)$: regularization term to penalize complexity

Regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

- T : number of leaves in the tree
- w : vector of output scores on leaves
- γ, λ : regularization hyperparameters

Explanation: This formulation balances model fit (through the loss L) and complexity (through Ω) to avoid overfitting.

2. Taylor Approximation

Using second-order Taylor expansion around $\hat{y}_i^{(t-1)}$:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

- $g_i = \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i}$
- $h_i = \frac{\partial^2 L(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$

Explanation: This approximation simplifies optimization using gradients (g_i) and Hessians (h_i).

3. Tree Structure Reformulation

Assume $f_t(x_i)$ maps x_i to leaf j with output score w_j . Let I_j be the set of samples in leaf j . Then:

$$\mathcal{L}^{(t)} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T$$

where:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

Explanation: The loss is grouped by leaves, allowing us to find optimal w_j for each leaf independently.

4. Optimal Weight for a Leaf

Set derivative of the loss with respect to w_j to zero:

$$\frac{\partial \mathcal{L}}{\partial w_j} = G_j + (H_j + \lambda) w_j = 0 \quad \Rightarrow \quad w_j^* = -\frac{G_j}{H_j + \lambda}$$

Explanation: This gives the best score for each leaf to minimize the total objective.

5. Optimal Objective Value After Adding Tree

Substitute w_j^* into the objective:

$$\mathcal{L}^{(t)} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

Explanation: This score helps evaluate split quality during tree construction.

6. Derivatives Summary

$$g_i = \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i} \quad (\text{gradient})$$

$$h_i = \frac{\partial^2 L(y_i, \hat{y}_i)}{\partial \hat{y}_i^2} \quad (\text{Hessian})$$

Explanation: These derivatives enable second-order optimization, improving training speed and accuracy.

7. Special Case: Squared Error Loss (Regression)

For $L(y_i, \hat{y}_i) = \frac{1}{2}(y_i - \hat{y}_i)^2$:

$$g_i = \hat{y}_i - y_i, \quad h_i = 1$$

$$w_j^* = \frac{\sum_{i \in I_j} (y_i - \hat{y}_i)}{N_j + \lambda}, \quad \mathcal{L}_j = \frac{\sum_{i \in I_j} (y_i - \hat{y}_i)^2}{N_j + \lambda}$$

where N_j is the number of samples in leaf j .

Explanation: For regression, gradient is the error, and the Hessian is constant, simplifying computation.